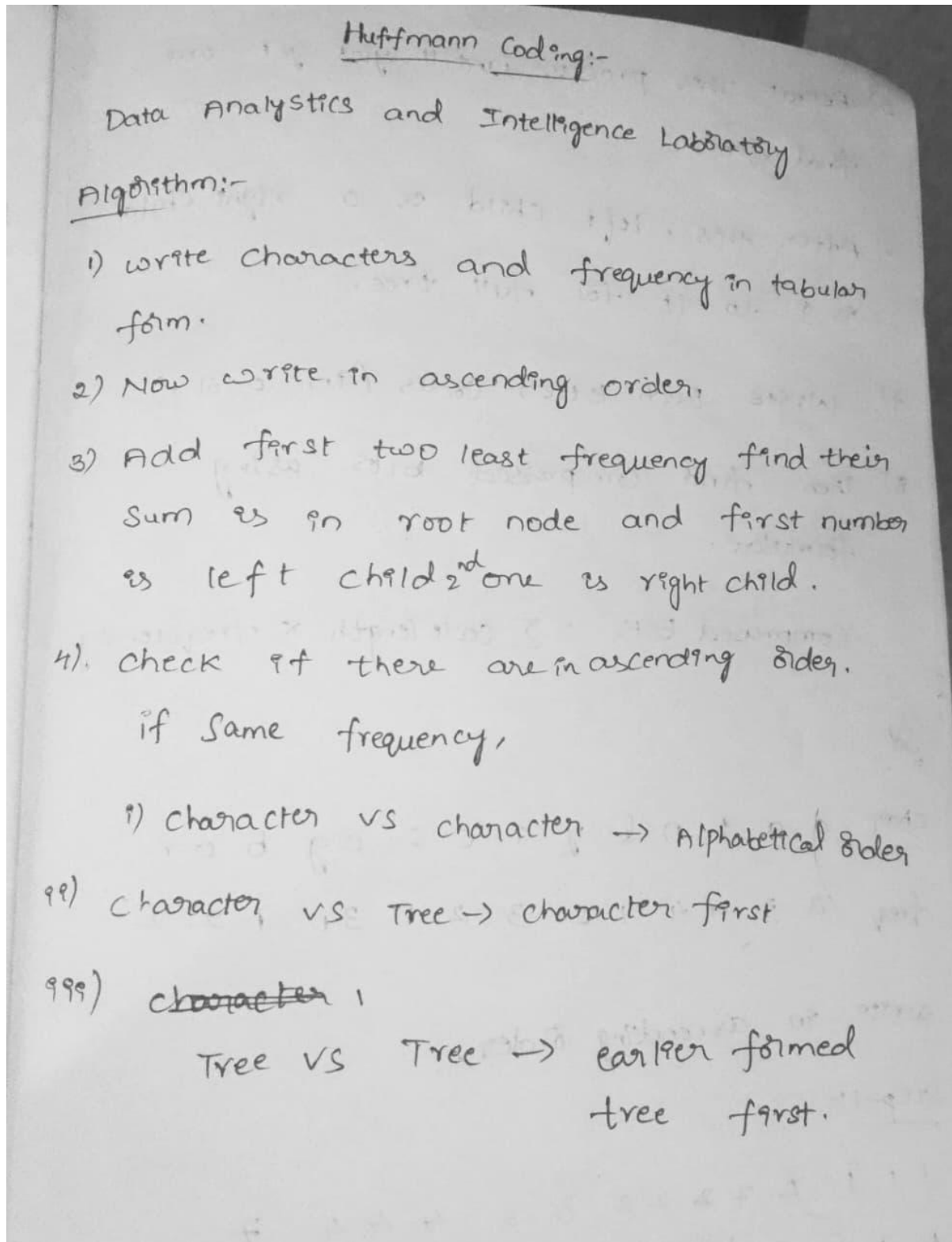


HUFFMAN CODING

ALGORITHM:



5) Repeat this process until you get one final tree.

6) After this, left child is 0 right child is 1 do it for full tree.

7) Write Each letter codes (in 0 1)

8) Then find compressed bits using formulae.

$$\text{Compressed bits} = \sum \text{code length} \times \text{frequency}$$

eg

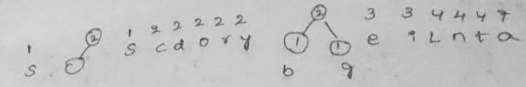
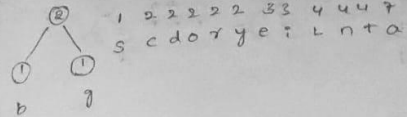
char d a t n l y i c s e g b o r

freq 2 7 4 4 4 2 3 2 1 3 1 1 2 2

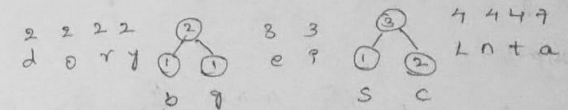
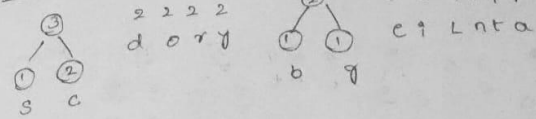
write in ascending order,

Step-1:-

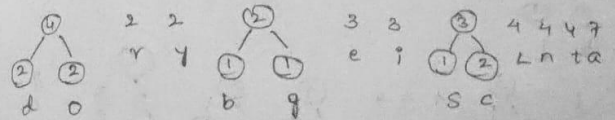
1 1 1 2 2 2 2 2 3 3 4 4 4 7
b g s c d o r y e i L n t a



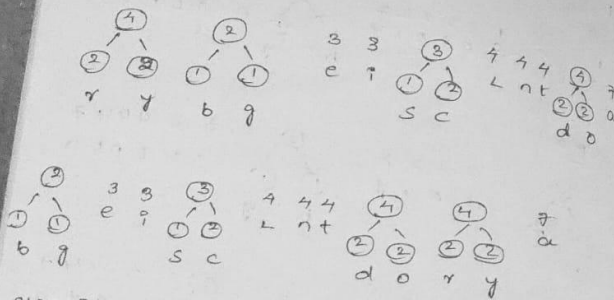
Step 2:-



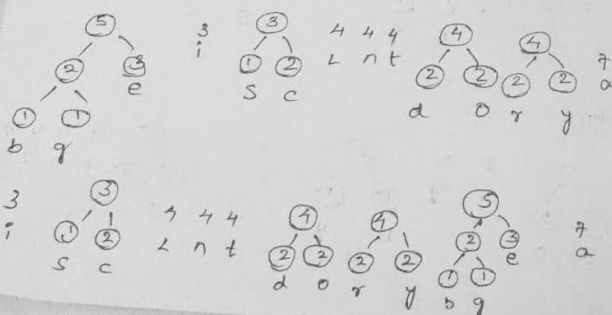
Step 3:-



Step-4:-



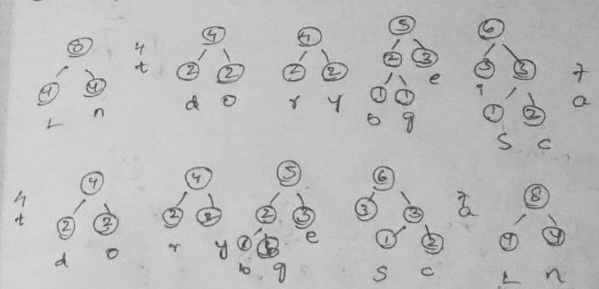
Step-5:-

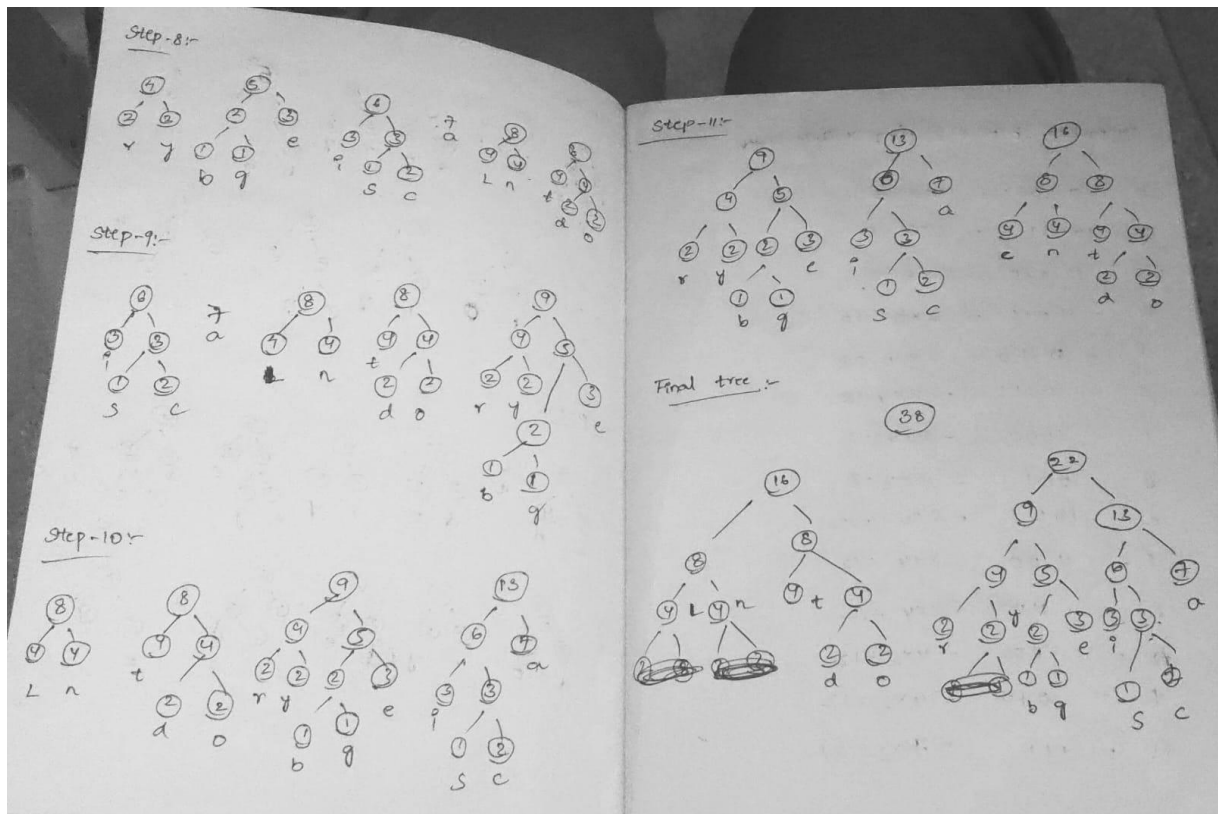


Step-6:-



Step-7:-





Compressed bits = $\sum \text{freq} \times \text{code length}$		original bit length = 38 KB = 304 bits	
b	10100	1x5=5	$\begin{aligned} \text{compressed bits} &= 5+5+5+8+8+8+8+10 \\ &+12+12+12+12+12+21 \\ &= 15+32+60+21+10 \\ &= 36+32+60+10 \\ &= 68+60+10 \\ &= 138 \end{aligned}$
q	10101	1x5=5	
s	11010	1x5=5	
c	11011	2x5=10	
d	0110	2x4=8	
o	0111	2x4=8	
r	1000	2x4=8	
y	1001	2x4=8	
e	1011	3x4=12	
p	1100	3x4=12	
L	1100	3x4=12	$\begin{aligned} \text{Solved bits} &= 304 - 138 \\ &= 166 \text{ bits} \end{aligned}$
n	000	4x3=12	
t	010	4x3=12	
a	111	7x3=21	

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 100

// Huffman Tree Node
struct Node {
    char data;
    int freq;
    struct Node *left, *right;
};

// Create new node
struct Node* createNode(char data, int freq) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = data;
    node->freq = freq;
    node->left = node->right = NULL;
    return node;
}

// Sort nodes in ascending order of frequency
void sort(struct Node* arr[], int n) {
    for(int i = 0; i < n-1; i++) {
        for(int j = i+1; j < n; j++) {
            if(arr[i]->freq > arr[j]->freq) {
                struct Node* temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
}
```

```

// Print Huffman Codes
void printCodes(struct Node* root, int code[], int top,
               int *totalBits, int *totalFreq) {

    if(root->left) {
        code[top] = 0;
        printCodes(root->left, code, top+1, totalBits, totalFreq);
    }

    if(root->right) {
        code[top] = 1;
        printCodes(root->right, code, top+1, totalBits, totalFreq);
    }

    // Leaf node
    if(!root->left && !root->right) {
        printf("%c : ", root->data);
        for(int i = 0; i < top; i++)
            printf("%d", code[i]);
        printf(" (freq=%d, length=%d)\n", root->freq, top);

        *totalBits += root->freq * top;
        *totalFreq += root->freq;
    }
}

int main() {

    char text[] = "DATA ANALYTICS AND INTELLIGENCE LABORATORY";
    int freq[256] = {0};

    // Step 1: Count frequency
    for(int i = 0; text[i]; i++) {
        if(text[i] != ' ')

```

```

// Step 1: Count frequency
for(int i = 0; text[i]; i++) {
    if(text[i] != ' ')
        freq[(int)text[i]]++;
}

struct Node* nodes[MAX];
int n = 0;

// Create initial nodes
for(int i = 0; i < 256; i++) {
    if(freq[i] > 0) {
        nodes[n++] = createNode((char)i, freq[i]);
    }
}

// Step 2-6: Build Huffman Tree
while(n > 1) {

    // Arrange in ascending order
    sort(nodes, n);

    // Pick two smallest
    struct Node* left = nodes[0];
    struct Node* right = nodes[1];

    // Create new internal node
    struct Node* newNode = createNode('$',
        left->freq + right->freq);

    newNode->left = left;
    newNode->right = right;

    // Replace first two with new node
    nodes[0] = newNode;
    nodes[1] = nodes[n-1];
    n--;
}

struct Node* root = nodes[0];

```

```

// Step 7-9: Generate codes & compute bits
int code[100], totalBits = 0, totalFreq = 0;

printf("Huffman Codes:\n\n");
printCodes(root, code, 0, &totalBits, &totalFreq);

printf("\nTotal Compressed Bits = %d\n", totalBits);

float avg = (float)totalBits / totalFreq;
printf("Average Code Length = %.2f bits\n", avg);

return 0;
}

```

OUTPUT:

Huffman Codes:

R : 0000 (freq=2, length=4)
D : 0001 (freq=2, length=4)
C : 0010 (freq=2, length=4)
O : 0011 (freq=2, length=4)
L : 010 (freq=4, length=3)
T : 011 (freq=4, length=3)
N : 100 (freq=4, length=3)
Y : 1010 (freq=2, length=4)
S : 10110 (freq=1, length=5)
B : 101110 (freq=1, length=6)
G : 101111 (freq=1, length=6)
E : 1100 (freq=3, length=4)
I : 1101 (freq=3, length=4)
A : 111 (freq=7, length=3)

Total Compressed Bits = 138

Average Code Length = 3.63 bits